Enterprise Leave Management System

Codebase Walkthrough Documentation

Java EE Web Application using Servlets, JSP, JDBC, MySQL, and Tomcat

Field	Details
Project	Enterprise Leave Management System with Data Analytics
Technology Stack	Java Servlets, JSP, JDBC, MySQL, HTML, CSS, JavaScript, Chart.js
Architecture	Three-tier MVC architecture
Roles	Employee, Manager, Administrator
Repository	https://github.com/gangadharygv-lang/elms

1. Purpose of This Document

This document explains the ELMS codebase from a developer and project documentation point of view. It describes the folder structure, MVC flow, database interaction, security handling, core modules, and how the implementation supports the project synopsis requirements.

The walkthrough is written so that the project can be explained during review, viva, or final demonstration without needing to open every source file.

2. Technology Stack

Layer	Technology	Responsibility
Presentation	JSP, HTML5, CSS3, JavaScript	Renders login, dashboards, leave forms, approvals, and admin screens.
Controller	Jakarta Servlets	Receives HTTP requests, validates input, invokes DAO classes, and forwards to JSP views.
Model	Java POJO classes	Represents users, leave requests, leave types, and leave balances.
Data Access	JDBC with PreparedStatement	Executes database queries safely and prevents SQL injection.
Database	MySQL 8	Stores users, leave records, balances, holidays, audit logs, and policies.
Server	Apache Tomcat 10.1	Hosts the Java web application.
Build	Maven WAR	Manages dependencies and packages the web application.

3. Project Folder Structure

```
elms/ pom.xml database/schema.sql src/main/resources/db.properties src/main/java/com/elms/ model/ User,
LeaveRequest, LeaveType, LeaveBalance dao/ DBConnection, UserDAO, LeaveRequestDAO, ReportDAO, AuditLogDAO
servlet/ LoginServlet, LeaveRequestServlet, ApprovalServlet, AnalyticsServlet filter/ AuthFilter util/
PasswordUtil, DateUtil, EmailUtil src/main/webapp/ WEB-INF/web.xml views/ JSP pages css/style.css js/main.js
```

The folder structure follows Maven web application conventions. Java source files are under src/main/java, configuration files are under src/main/resources, and JSP/CSS/JavaScript files are under src/main/webapp.

4. MVC Architecture Walkthrough

MVC Part	Files	Explanation
Model	User.java, LeaveRequest.java, LeaveType.java, LeaveBalance.java	Plain Java classes store application data and expose getter/setter methods.
View	login.jsp, dashboard.jsp, applyLeave.jsp, approvalQueue.jsp, analytics.jsp, users.jsp	JSP pages render browser screens using request/session attributes.
Controller	LoginServlet, DashboardServlet, LeaveRequestServlet, ApprovalServlet, AdminUsersServlet	Servlets process requests and decide which JSP to show or which action to perform.
DAO	UserDAO, LeaveRequestDAO, LeaveBalanceDAO, ReportDAO, AuditLogDAO	DAO classes isolate SQL queries and database operations from servlet code.

General Request Flow

```
Browser request -> AuthFilter checks active session -> Servlet validates request data -> DAO executes SQL
using JDBC -> Servlet stores results in request attributes -> JSP renders final HTML response
```

5. Database Design

Table	Purpose
departments	Stores department master data.
users	Stores employee, manager, and admin accounts with role and manager mapping.
leave_types	Defines leave categories such as CL, EL, ML, MAT, PAT, CO, and LOP.
leave_requests	Stores leave applications, dates, status, remarks, and action timestamps.
leave_balances	Tracks yearly entitlement, taken days, remaining days, and carry-forward values.
public_holidays	Stores holidays excluded from working-day leave calculations.
audit_log	Stores user and leave actions for accountability and compliance.

The database schema is defined in database/schema.sql. It creates the elms_db database, tables, foreign keys, indexes, demo users, leave types, leave balances, and public holidays.

6. Authentication and Authorization

File	Responsibility
LoginServlet.java	Accepts email/password, hashes password using SHA-256, authenticates through UserDAO, creates HttpSession.
PasswordUtil.java	Generates SHA-256 hash for secure credential checking.
AuthFilter.java	Blocks unauthenticated access and checks role restrictions for /admin and /manager paths.
LogoutServlet.java	Invalidates the current session and redirects to login.

```
Login flow: 1. User submits email and password. 2. PasswordUtil.sha256(password) creates a hash. 3.
UserDAO.authenticate(email, hash) checks active user record. 4. LoginServlet stores User object in
```

```
HttpSession. 5. AuditLogDAO writes a LOGIN entry.
```

7. Leave Request Module

The leave request module allows employees to apply for leave, view balances, upload supporting documents, and cancel pending requests. The main controller is LeaveRequestServlet.

Step	Implementation
Load form	LeaveRequestServlet.doGet loads active leave types and current user balances.
Validate dates	Rejects past dates, invalid date ranges, and multi-day half-day requests.
Conflict check	LeaveRequestDAO.hasOverlap prevents duplicate pending or approved requests in the same date range.
Working days	DateUtil.calculateWorkingDays excludes weekends and public holidays.
Balance check	LeaveBalanceDAO.getAvailableBalance confirms sufficient leave before insert.
Attachment	Multipart upload accepts PDF/JPG/JPEG and stores files in a temporary upload folder.
Audit	AuditLogDAO writes SUBMITTED action after successful request creation.

8. Approval Workflow

Managers use ApprovalServlet and approvalQueue.jsp to approve or reject pending requests from their reportees. Each decision requires remarks.

```
Approval transaction: 1. Manager submits approve or reject. 2. LeaveRequestDAO locks the request using SELECT
... FOR UPDATE. 3. Request status changes to APPROVED or REJECTED. 4. If approved, leave_balances.days_taken
increases and days_remaining decreases. 5. AuditLogDAO records the manager decision. 6. Transaction commits.
On error, it rolls back.
```

This transaction protects data consistency. A leave request cannot be approved without updating the corresponding leave balance.

9. Admin User Management

Feature	Files	Explanation
Create user	AdminUsersServlet, UserDAO, users.jsp	Admin creates employee, manager, or admin accounts with temporary password.
Edit user	AdminUsersServlet.updateUser, UserDAO.updateBasic	Admin can update code, name, email, role, active flag, and optional password.
Deactivate user	AdminUsersServlet.toggleUser, UserDAO.setActive	Admin can activate or deactivate accounts. Self-deactivation is blocked.
Audit trail	AuditLogDAO	CREATED, UPDATED, ACTIVATED, and DEACTIVATED actions are recorded.

10. Reports and Analytics

The analytics module uses ReportDAO to aggregate leave data and AnalyticsServlet to prepare chart-ready JSON. The analytics.jsp page displays visual dashboards using Chart.js.

Chart	Data Source	Purpose
Status doughnut chart	ReportDAO.countByStatus	Shows pending, approved, rejected, and cancelled distribution.
Leave type pie chart	ReportDAO.countByLeaveType	Shows approved leave by leave category.
Department bar chart	ReportDAO.countByDepartment	Shows department-wise utilization.
Monthly trend line chart	ReportDAO.monthlyApprovedTrend	Shows month-wise approved leave trend.

11. Audit Trail Implementation

The audit trail strengthens accountability. Important user and leave operations are saved into the audit_log table with entity type, entity id, action, performer, old value, new value, IP address, and timestamp.

Action	Recorded From
LOGIN	LoginServlet
SUBMITTED	LeaveRequestServlet
CANCELLED	MyRequestsServlet via LeaveRequestDAO
APPROVED / REJECTED	ApprovalServlet via LeaveRequestDAO
CREATED / UPDATED / ACTIVATED / DEACTIVATED	AdminUsersServlet

12. JSP View Pages

JSP Page	Purpose
login.jsp	Login screen for all roles.
dashboard.jsp	Employee dashboard with leave balances and recent requests.
applyLeave.jsp	Leave application form with dynamic balance and duration hints.
myRequests.jsp	Employee request history and pending cancellation.
approvalQueue.jsp	Manager approval queue with remarks.
analytics.jsp	Chart.js analytics dashboard and recent activity table.
admin/users.jsp	Admin create, edit, activate, and deactivate user accounts.
partials-nav.jsp	Shared role-based navigation bar.

13. Security and Data Integrity

Concern	Implementation
Authentication	Session-based login using HttpSession.
Password security	SHA-256 hashing through PasswordUtil.
SQL injection prevention	All database operations use PreparedStatement.
Role-based access	AuthFilter restricts admin and manager routes.
Balance consistency	Approval flow uses a JDBC transaction and row lock.
Auditability	AuditLogDAO records important operations.
Upload safety	Attachment upload allows only PDF/JPG/JPEG and max size is configured by MultipartConfig.

14. Synopsis Feature Mapping

Synopsis Requirement	Implementation Status
Role-based login for Employee, Manager, Admin	Implemented through LoginServlet, User.Role, AuthFilter, and role-based navigation.
Leave request submission and tracking	Implemented through LeaveRequestServlet, MyRequestsServlet, and leave request JSP pages.
Approval/rejection workflow	Implemented through ApprovalServlet and LeaveRequestDAO transactions.
Admin dashboard and user management	Implemented with create, edit, activate, and deactivate user functions.
Reports and analytics	Implemented with Chart.js charts and ReportDAO aggregation queries.
Audit trail	Implemented using audit_log table and AuditLogDAO.
Attachment upload	Implemented in LeaveRequestServlet using MultipartConfig.
Email notification	EmailUtil hook exists; SMTP delivery is a future configuration item.
Team calendar and escalation	Not implemented in current version; suitable for future scope.

15. Demonstration Flow

```
Recommended demo sequence: 1. Start MySQL and Tomcat. 2. Open http://localhost:8080/elms. 3. Login as
employee@elms.local / password123. 4. Apply for leave and observe balance validation. 5. Login as
manager@elms.local and approve or reject the request. 6. Login as admin@elms.local. 7. Open Analytics to show
charts. 8. Open Users to edit or deactivate an account. 9. Check audit_log table to prove actions were
recorded.
```

16. Viva Explanation Points

Question	Suggested Answer
Why MVC?	MVC separates UI, request handling, business/data logic, making the project easier to maintain.
Why JDBC PreparedStatement?	It safely binds input parameters and reduces SQL injection risk.
How is leave balance protected?	Approval runs inside a transaction. Status update and balance deduction succeed or fail together.
How are roles handled?	User.Role defines EMP, MGR, ADMIN, and AuthFilter restricts protected URLs.
How does analytics work?	ReportDAO executes aggregate SQL queries and Chart.js renders the returned data visually.
How is accountability maintained?	AuditLogDAO inserts action records into audit_log for important events.

17. Conclusion

The ELMS codebase implements the main features promised in the synopsis: role-based leave workflow, approval handling, leave balance tracking, admin user management, analytics, and audit logging. The project demonstrates a practical Java EE MVC application with MySQL-backed persistence and a maintainable modular structure.